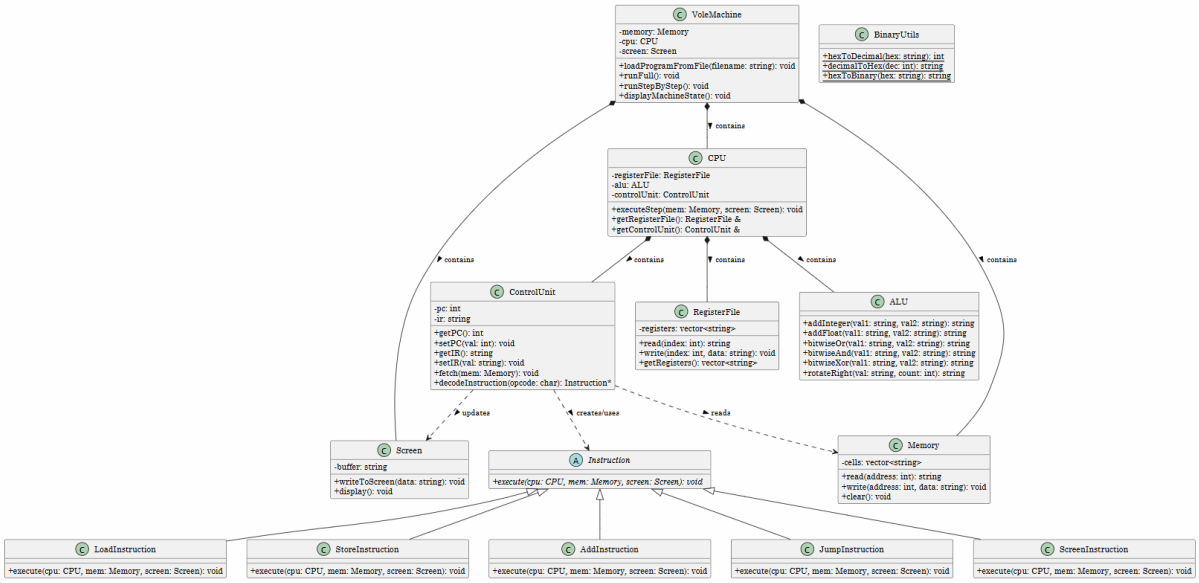
Design 3

UML Class Diagram



PlantUML Code

```
@startuml
skinparam classAttributeIconSize 0

class VoleMachine {
  - memory: Memory
  - cpu: CPU
  - screen: Screen
  + loadProgramFromFile(filename: string): void
  + runFull(): void
  + runStepByStep(): void
  + displayMachineState(): void
}

class CPU {
  - registerFile: RegisterFile
  - alu: ALU
  - controlUnit: ControlUnit
  + executeStep(mem: Memory, screen: Screen): void
  + getRegisterFile(): RegisterFile &
  + getControlUnit(): ControlUnit &
}

class RegisterFile {
  - registers: vector<string>
  + read(index: int): string
  + write(index: int, data: string): void
  + getRegisters(): vector<string>
}

class ALU {
  + addInteger(val1: string, val2: string): string
  + addFloat(val1: string, val2: string): string
  + bitwiseOr(val1: string, val2: string): string
  + bitwiseAnd(val1: string, val2: string): string
  + bitwiseXor(val1: string, val2: string): string
  + rotateRight(val: string, count: int): string
}

class ControlUnit {
  - pc: int
  - ir: string
  + getPC(): int
  + setPC(val: int): void
  + getIR(): string
  + setIR(val: string): void
  + fetch(mem: Memory): void
  + decodeInstruction(opcode: char): Instruction*
}

abstract class Instruction {
  + {abstract} execute(cpu: CPU, mem: Memory, screen: Screen): void
}

class LoadInstruction {
  + execute(cpu: CPU, mem: Memory, screen: Screen): void
}

class StoreInstruction {
  + execute(cpu: CPU, mem: Memory, screen: Screen): void
}

class AddInstruction {
  + execute(cpu: CPU, mem: Memory, screen: Screen): void
}
```

```
class JumpInstruction {
  + execute(cpu: CPU, mem: Memory, screen: Screen): void
}

class ScreenInstruction {
  + execute(cpu: CPU, mem: Memory, screen: Screen): void
}

class Memory {
  - cells: vector<string>
  + read(address: int): string
  + write(address: int, data: string): void
  + clear(): void
}

class Screen {
  - buffer: string
  + writeToScreen(data: string): void
  + display(): void
}

class BinaryUtils {
  + {static} hexToDecimal(hex: string): int
  + {static} decimalToHex(dec: int): string
  + {static} hexToBinary(hex: string): string
}

VoleMachine *-- Memory : contains >
VoleMachine *-- CPU : contains >
VoleMachine *-- Screen : contains >
CPU *-- RegisterFile : contains >
CPU *-- ALU : contains >
CPU *-- ControlUnit : contains >
ControlUnit ..> Instruction : creates/uses >
Instruction <|-- LoadInstruction
Instruction <|-- StoreInstruction
Instruction <|-- AddInstruction
Instruction <|-- JumpInstruction
Instruction <|-- ScreenInstruction
ControlUnit ..> Memory : reads >
ControlUnit ..> Screen : updates >

@enduml
```